Authentication and Authorization in ASP.NET Web API

Back to: ASP.NET Web API Tutorial For Beginners and Professionals

Authentication and Authorization in ASP.NET Web API

In this article, I will talk about **Authentication and Authorization in ASP.NET Web API** Here, I will give you an overview of Authentication and Authorization in ASP.NET Web API. From the next article onwards, we will discuss the practical implementation of Authentication and Authorization in ASP.NET Web API. Please read the following points.

What are Authentication and Authorization in Web API?

Once you create a Web API Service, the most important thing you need to take care of is security which means you need to secure your Web API Service. In Web API, there are two ways to secure your Web API Service, Authentication and Authorization. Authentication and Authorization are critical in building secure web applications, including ASP.NET Web API applications. Authentication is the process of verifying the identity of a user. At the same time, Authorization determines what actions and resources the Authenticated user is allowed to access based on their identity and roles. Authentication is performed first and then Authorization.

Authentication in Web API:

Authentication is the process of identifying the user. For example, one user's key, password, login is with the username and password, and the same user's fingerprint and password is authentication series. Common authentication mechanisms for Web APIs include:

- **Basic Token Authentication**: Clients include a token (e.g., JWT or OAuth token) in the request headers, which the server validates.
- **Basic Authentication**: Clients send a username and password with each request, typically encoded in Base64 format.
- **API Key Authentication**: Clients include an API key in the request, validated on the server.
- **OAuth 2.0**: OAuth 2.0 is a framework for securing APIs. It allows clients to obtain access tokens used to authenticate requests.
- **Custom Authentication**: You can implement custom authentication logic based on your requirements.
- **Windows Authentication**: If you are running your application in a Windows environment, you can use Windows Authentication to authenticate users based on their Windows credentials.

Authorization in Web API:

Authorization is deciding whether the authenticated user can perform an action or a specific resource (Web API Resource). For example, among all authenticated users, two permission to get a resource but does not have permission to create a resource. So, once a user is authenticated, you can implement authorization by defining Roles and Policies or using attributes to your controllers and actions.

- **Role-Based Authorization**: You can use role-based authorization to restrict access to specific resources or actions based on the user's role. Roles are assigned to users during authentication. User's roles determine access rights. You can decorate controllers or actions with the **[Authorize(Roles = "Role")]** attribute.
- **Policy-Based Authorization**: Policies define more complex authorization rules than just roles or actions. Policies can be based on a combination of roles, claims, or custom requirements. Policies are defined with specific authorization requirements, and enquiries are processed using the **[Authorize(Policy = "PolicyName")]** attribute.
- **Claim-Based Authorization**: Resource access is based on the user's claims in claims-based authorization. You can use the [Authorize] attribute with specific claims as parameters.
- **Custom Authorization**: If the built-in authorization methods do not meet your requirements, you can implement custom authorization logic by extending the authorization framework or implementing custom middleware.

Authentication in ASP.NET Web API:

The ASP.NET Web API Service assumes that the authentication process should happen in the host Server, and we generally host the ASP.NET Web API Service as IIS. The IIS Server uses the **HTTP Modules** to check user's authentication. You can configure your project to use any of the built-in authentication modules available in IIS or ASP.NET or create your own HTTP module to perform custom authentication.

When the host IIS Server authenticates the user, it generally creates a principal object (i.e. **IPrincipal** object) under which the code runs. The IIS Server then attaches the principal object to the current thread. In .NET, IIS Server attaches the **Principal Object** to the **System.Threading.Thread.CurrentPrincipal** property.

If you are confused about how the Principal object and the Identity object the principal object is attached to the current thread, don't worry, we will discuss all these things in greater detail in our upcoming articles. This article will give you an overview of how authentication and authorization happen in Web API services.

Understanding Principal Object

The Principal interface contains the property, the **Identity** object, which contains the information about the user, and the other one is the **IsInRole** property, which is a boolean property. The boolean property is set to true if the user is assigned any roles mentioned. The following diagram shows the Principal interface definition.

```
namespace System.Security.Principal
{
    //
    // Summary:
    //     Defines the basic functionality of a principal object.
    [ComVisible(true)]
    public interface IPrincipal
    {
        [MethodImpl(MethodImplOptions.InternalCall)]
        [SecurityCritical]
        [SecuritySafeCriticalAttribute]
        [__DynamicIntrinsicAttribute]
        [ReadOnly(true)]
        IIdentity Identity { get; }

        [MethodImpl(MethodImplOptions.InternalCall)]
        [SecurityCritical]
        [SecuritySafeCriticalAttribute]
        [__DynamicIntrinsicAttribute]
        [ReadOnly(true)]
        bool IsInRole(string role);
    }
}
```

Let's look at the Identity interface definition, which contains the user's information.

```
namespace System.Security.Principal
{
    //
    // Summary:
    //     Defines the basic functionality of an identity object.
    [ComVisible(true)]
    public interface IIdentity
    {
        [MethodImpl(MethodImplOptions.InternalCall)]
        [SecurityCritical]
        [SecuritySafeCriticalAttribute]
        [__DynamicIntrinsicAttribute]
        [ReadOnly(true)]
        string Name { get; }

        [MethodImpl(MethodImplOptions.InternalCall)]
        [SecurityCritical]
        [SecuritySafeCriticalAttribute]
        [__DynamicIntrinsicAttribute]
        [ReadOnly(true)]
        bool IsAuthenticated { get; }
    }
}
```

The Identity Object which is a property of the Principal Object, contains three properties, i.e. **Name** (string type), **AuthenticationType** (string type), and **IsAuthenticated** (boolean type). If the user is authenticated, the **IsAuthenticated** property will return true; otherwise, it will return **false**. The **Name** property of the Identity object will store the user's name. The **AuthenticationType** property stores the Kerberos or Basic or Digest or Negotiate or Kerberos. Similarly, the **AuthenticationType** property stores the type of authentication used to identify the user.

The Current.Identity and IPrincipalIdentity classes generally implement the Identity interface. Our upcoming article will discuss these three classes implement the Identity interface.

HTTP Message Handlers for Authentication in Web API:

Instead of using the host i.e. IIS Server, where the ASP.NET Web API service is hosted, for authentication, you can also write the authentication logic into the HTTP **Message Handler**. In that case, the **HTTP Message Handler** will check the incoming HTTP request for authenticating the user and then set the Principal Object.

Differences Between HTTP Message Handler Over HTTP Module:

An HTTP **Module** sees all the incoming requests that go through the ASP.NET pipeline, whereas a message handler only sees those requests assigned to the ASP.NET Web API Service.

It is also possible to select a specific **HTTP Message Handler** and then use it for authentication for a specific route. The **HTTP Modules** are specific to IIS, whereas the **HTTP Message Handlers** can be used with both **Web Hosting** (within a service) and **self-hosting** (with or outside IIS).

The HTTP Modules participate in IIS logging, auditing, and so on. Generally, if you don't want to support self-hosting, then HTTP Modules is a better option, but if you want to support self-hosting, then HTTP Message Handler is a better option.

Setting the Principal Object

If you are using the implement your own custom logic for authenticating the user, then you can set the principal object at two places, which are as follows:

- **ThreadCurrentPrincipal**: This is the standard way to set the Principal object in .NET.
- **HttpContext.Current.User**: This property is specific to ASP.NET.

The following image shows how to create and set the principal object with the current thread. I am showing you both options to set the Principal object here.

```
//Input: Create Identity Object
var identity = new GenericIdentity(username);

//Input: Create Principal Object
var principal = new GenericPrincipal(identity, null);

//Option1:
Thread.CurrentPrincipal = principal;

//Option2:
if (HttpContext.Current != null)
{
    HttpContext.Current.User = principal;
}
```

Suppose you are going to host the Web API service in IIS. In that case, you need to set the principal object in both places because of the security concerns, i.e. security becomes inconsistent when Thread.CurrentPrincipal property value is not. To ensure your code is host-agnostic (i.e. to support web hosting and self-hosting), you must check for null before assigning the Principal object to the **HttpContext.Current**, as shown in the above image.

Authentication in ASP.NET Web API

The Authentication Process will happen before executing the Controller Action Method, allowing you to decide whether you want to grant access to that resource.

We can implement this in ASP.NET Web API using the Authorization filters, which will be executed before the controller action method is executed. So, if the request is not authorized for that specific resource, then the filter returns an error response to the client without executing the controller action method. The following diagram explains the above.

Authentication and Authorization in Web API

Using the [Authorize] Attribute

The ASP.NET Web API Framework provides a built-in authorization filter attribute, i.e. **AuthorizeAttribute**, and you can use this built-in filter attribute to secure your Web API service or controller. If no it restricts the HTTP status code **401 Unauthorized** without invoking the controller action method. You can apply the **Authorize** filter globally, or at the controller or action levels.

At Globally

Suppose you want to check the authentication for all the Web API controllers. In that case, it is better to add the **AuthorizeAttribute** filter to the global filter list within the **Register** method of the **WebApiConfig** class, as shown in the image below.

```
public static class WebApiConfig
{
    public static void Register(HttpConfiguration config)
    {
        //
        config.Filters.Add(new AuthorizeAttribute());

        // Web API routes
        config.MapHttpAttributeRoutes();

        //
        config.Routes.MapHttpRoute(
            name: "DefaultApi",
            routeTemplate: "api/{controller}/{id}",
            defaults: new { id = RouteParameter.Optional }
        );
    }
}
```

At Controller Level

Suppose you want to provide authentication for all the action methods of a specific controller. In that case, adding the Authorize filter at the controller level is better and recommended, as shown in the image below.

```
[Authorize]
public class ValuesController : ApiController
{
    // GET api/values
    [HttpGet]
    [ActionName("GetAllValues")]
    public IEnumerable<string> GetAllValues()
    {
        return new string[] { "value1", "value2" };
    }

    // GET api/values/5
    [HttpGet]
    [ActionName("GetValueById")]
    public string GetValueById(int id)
    {
        return "value";
    }

    // POST api/values
    [HttpPost]
    [ActionName("PostNewValue")]
    public void PostNewValue([FromBody]string value)
    {
    }

    // PUT api/values/5
    [HttpPut]
    [ActionName("PutValueUpdateById")]
    public void PutValueUpdateById(int id, [FromBody]string value)
    {
    }

    // DELETE api/values/5
    [HttpDelete]
    [ActionName("DeleteValueById")]
    public void DeleteValueById(int id)
    {
    }
}
```

At Action Level

Suppose you want to provide authentication for specific action methods of a controller. In that case, adding the Authorize filter attribute to the action method that requires authentication is better, as shown in the image below.

```
[AllowAnonymous]
public class ValuesController : ApiController
{
    // GET api/values
    [HttpGet]
    [ActionName("GetAllValues")]
    public IEnumerable<string> GetAllValues()
    {
        return new string[] { "value1", "value2" };
    }

    // GET api/values/5
    [HttpGet]
    [ActionName("GetValueById")]
    public string GetValueById(int id)
    {
        return "value";
    }

    // POST api/values
    [HttpPost]
    [ActionName("PostNewValue")]
    [Authorize]
    public void PostNewValue([FromBody]string value)
    {
    }

    // PUT api/values/5
    [HttpPut]
    [ActionName("PutValueUpdateById")]
    public void PutValueUpdateById(int id, [FromBody]string value)
    {
    }

    // DELETE api/values/5
    [HttpDelete]
    [ActionName("DeleteValueById")]
    public void DeleteValueById(int id)
    {
    }
}
```

Another way of doing this is to restrict the controller by decorating the controller with the **Authorize** filter attribute and then allow the anonymous access to the Web API Service methods by using the **AllowAnonymous** attribute. In the above example, the **AllowAnonymous** attribute, in the Values example, the Post method is restricted, but the Get method allows anonymous access.

As of now, we have discussed two things. If we want to check the authentication before invoking the action method, we need to use the built-in **Authorize** Filter Attribute. If we want any action method to be accessible by anonymous users, then we need to use the built-in filter attribute with the **AllowAnonymous** attribute. Along the way, we can limit access to specific users or users with specific roles.

Reported by Usama

Reported by Rizwan.

Note: The point to remember here is that the **AuthorizeAttribute** filter in ASP.NET Web API is located in the **System.Web.Http.Filters** namespace, and in ASP.NET MVC, it is located in **AuthorizeAttribute** filter located in the **System.Web.Mvc** namespace, which is not compatible with ASP.NET Web API controllers.

Authorization Inside a Controller Action

In some scenarios, you might allow a request to proceed, but you need to change the behavior based on the principal. For example, the information you will return from the action depends on the user's role.

In the next article, I will discuss how to implement **ASP.NET Web API Basic Authentication** with an example. In this article, I try to give you an overview of **Authentication and Authorization in ASP.NET Web API**. I hope this article will help you with your needs. Let me know your thoughts about this article. Your feedback is very important to improve the quality of the article. If you want to learn C# in depth, please look at our **C# In Depth with Architecture** tutorials. We believe you will enjoy this course.

Dot Net Tutorials
About the Author: Pranaya Rout
Pranaya Rout has published more than 3,000 articles in his 12 year career. Pranaya Rout has very good knowledge on .Net Framework, ASP.Net MVC, ASP.Net Web API, ASP.Net Core, C#, ASP.Net Core Web API, C# Core, ASP.NET CORE, SQL Server, HTML5, Ajax, Bootstrap, ASP.NET Core Security, Microservices, Design Patterns, Entity Framework, Entity Framework Core, LINQ, etc.

5 thoughts on "Authentication and Authorization in ASP.NET Web API"

Vikas says:
FEBRUARY 14, 2019 AT 12:03 PM
Best article, Many thanks...
Reply
Jasmine
APRIL 30, 2019 AT 1:38 PM
Best Article and great explanation in simple English for beginners to understand. Thanks! This article was a blessing to me at work! Cheers!
Reply
Jasmine Julian
MAY 1, 2019 AT 12:09 PM
Great article! Thanks!
I want to do a comment if you allow me.
When we plan to create an enterprise level application, we especially want to take care of authentication and authorization. These are two techniques if used well makes our application secure, in our case makes our Web API secure.
An application program interface (API) is a set of routines, protocols, and tools for building software applications. Basically an API specifies how software components should interact. Additionally, APIs are used when programming graphical user interface (GUI) components.
Authentication
Authentication is all about the identity of an end user. It's about validating the identity of a user who is accessing our system, that is authenticated enough.
Credentials can be in the form of a username and password. Well, just Basic Authentication technique to understand how we can achieve authentication in WebAPI.
Authorization
Authorization should be considered as a second step after authentication to achieve security. Authorization means what an authenticated user can do with our system. Authorization is the process of granting access to a resource based on permissions. This could be achieved by using permissions for an end user which are authorized to perform or can be achieved through providing a secure token, using which an end user can have access to other services or resources.
Remaining Session
RESTful services work on a stateless protocol i.e. HTTP. We can achieve maintaining session in Web API through cookie-based authentication technique. An authenticated user will be allowed to access resources for a particular period of time and can be authenticated for subsequent requests using session cookies. Session cookies are not supported by default in Web API but WebAPI as RESTful service may need to implement logging/auditing for a user, to maintain session for the user, to provide roles and permissions for their user. All these features could be achieved using basic authentication and token based authentication.
Pros and Cons of Basic Authentication
Basic authentication has its own pros and cons. It's advantageous when it comes to implementation. It is very easy to implement. It is supported by all browsers. It's widely browsers and has become an authentication mechanism in RESTful. Only HTTPS. It has disadvantages like sending user credentials in plain text, sending user credentials inside request headers, no proxies to flash. Time to live is not considerable and time consumed is called. No session is maintained and a user cannot log out and logged in through basic authentication. It is very prone to CSRF (Cross Site Request Forgery).
Regarding
Reply
Yamaha
OCTOBER 2, 2021 AT 8:52 AM
Thanks! It's very helpful.
Reply
CONTRACTS
OCTOBER 14, 2021 AT 8:49 PM
Hello, can ask Why The HTTP Methods is better option if it doesn't support self hosting? Or the Differences HTTP Message Handler over HTTP Module part? It means whenever access the web they must to go through the authentication? Thank you.
Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name * Email * Website

Post Comment